

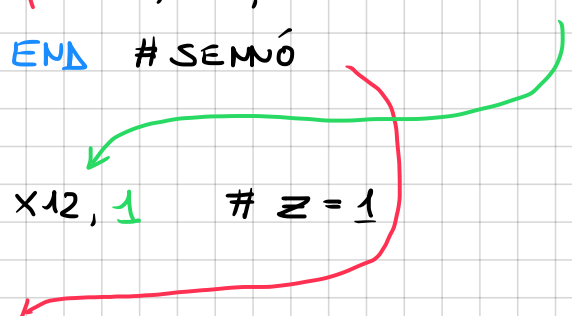
IN C:

```
IF (x == y) {  
    z = 1;  
}
```

=>

IN ASSEMBLY

```
BEQ x10, x11, THEN # SE x == y  
J END # SEMO  
THEN:  
LI x12, 1 # z = 1  
END:
```



ATTENZIONE A QUELLO CHE È FUORI DALL'IF!

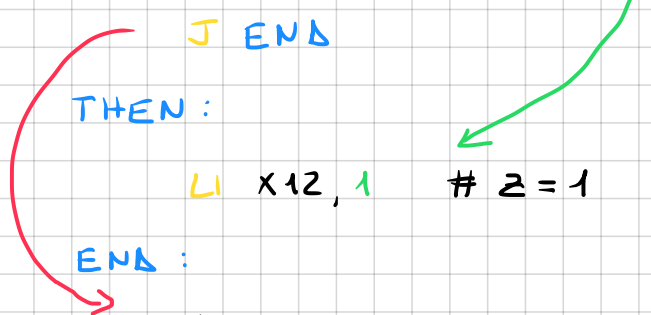
IN C:

```
IF (x == y) {  
    z = 1;  
}  
z = 2;
```

=>

ESCO

```
BEQ x10, x11, THEN # SE x == y  
LI x12, 2  
J END  
THEN:  
LI x12, 1 # z = 1  
END:
```



LI x12, 2 SE LO AVESSI MESSO QUI
SAREBBE STATO ESEGUITO
LO STESSO DOPO THEN, QUINDI
NEL VERO,

MA NOI VOGLIAMO CHE VIENE ESEGUITO SOLO SE È
FALSO.

PER AVVICINARCI AL LINGUAGGIO C LO SCRIVIAMO COSÌ:

```
BEQ x10, x11, THEN  
ELSE: # ANCHE SE NON SERVE A NULLA LO FACCIAMO  
      # PER LEGGIBILITÀ.  
LI x12, 2  
J END  
THEN:  
LI x12, 1  
END:
```

IN C :

```
IF (x == y) {  
    z = 1;  
}  
ELSE {  
    z = 2;  
}  
z = z + z;
```

=>

IN ASSEMBLY

```
BEQ x10, x11, THEN # SE x == y  
ELSE:  
    LI x12, 2 # z = 2  
    J END  
THEN:  
    LI x12, 1 # z = 1  
END:  
    ADD x12, x12, x12 # z = z + z
```

IN ASSEMBLY NON ABBIAMO > DIRETTO QUINDI POSSIAMO INVERTIRE LA CONDIZIONE CON <= O VERO NOT > :

IN C :

```
IF (x == y) {  
    IF (x > 0) z = 1;  
    ELSE z = 2;  
}  
ELSE {  
    z = 3;  
}
```

=>

IN ASSEMBLY :

```
BEQ x10, x11, THEN1  
ELSE1:  
    LI x12, 3  
    J END  
THEN1:  
    BLEZ x10, THEN2  
ELSE2:  
    LI x12, 1  
    J END  
THEN2:  
    LI x12, 2  
END:
```

SENZA FARLO APPOSTA
QUESTO CODICE È +
EFFICIENTE DI QUELLO NEL
PROF. (HA MESSO UN
SALTO J IN PIÙ)

UNO SWITCH IN C :

```
INT x = 2, y;
```

```
SWITCH (x) {
```

```
    CASE 0 : => OGNI CASE  
                DIVENTA UN BRANCH
```

```
        y = 10; CONDIZIONATO
```

```
    BREAK;
```

```
    CASE 1:
```

```
        y = 20;
```

```
    BREAK;
```

```
    ...
```

```
    DEFAULT :
```

```
        y = 0;
```

```
}
```

OGNI CASO PORTA

POI ALLA FINE, PER

NON ESEGUIRE IL

CODICE SUCCESSIVO

o IN MEZZO.

IN ASSEMBLY :

```
LI T0, 2    # x = 2
```

```
LI T1, 0    # y = 0
```

```
BEQ T0, X0, CASE0 # CASE 0 :
```

```
LI T2, 1    # UTILIZZO UN REGISTRO PER  
                I CONFRONTI
```

```
BEQ T0, T2, CASE1 # CASE 1 :
```

```
LI T2, 2
```

```
BEQ T0, T2, CASE2 # CASE 2 :
```

```
...
```

```
J DEFAULT
```

```
CASE0 :
```

```
LI T1, 10    # y = 10
```

```
J END
```

```
CASE1 :
```

```
LI T1, 20
```

```
J END
```

```
...
```

```
DEFAULT :
```

```
LI T1, 0
```

```
END :
```

ORA ABBIAMO I CICLI WHILE :

IN C :

```
INT x = 0;
```

```
WHILE (x < 5) {
```

```
    x++;
```

```
}
```

=>

```
LOOP :
```

```
LI T0, 0    # x = 0
```

```
LI T1, 5    # NH CICLO
```

```
BGE T0, T1, EXIT
```

```
ADDI T0, T0, 1 # x++
```

```
J LOOP
```

```
EXIT :
```

SALTIAMO IN
AVANTI SOLO 1
VOLTA SOLA
(I SALTI IN AVANTI SONO
INEFFICIENTI)

IL **DO-WHILE** INVECE È MOLTO EFFICIENTE PERCHÉ FA SOLO SALT1 ALL' INDIETRO:

IN C:

```
INT x = 0
```

```
DO {  
    x++;  
} WHILE (x < 5);
```

VIENE ESEGUITO
ALMENO 1 VOLTA

IN ASSEMBLY

```
LJ TO, 0 # x = 0
```

```
LJ T1, 5 # DIM LOOP
```

LOOP:

```
ADD TO, TO, 1 # x++
```

```
BLT TO, T1, LOOP # SE x < 5 TORNA  
INDIETRO
```

EXIT:

UN **FOR** INVECE È SEMPLICEMENTE UN **WHILE** CAMUFFATO:

IN C:

```
FOR (INT i = 0, i < 5, i++) {  
    // DO SOMETHING  
}
```

PROPRIO COME UN
WHILE NORMALE SALTA IN
AVANTI SOLO QUANDO IL CICLO
È FINITO.

IN ASSEMBLY

```
LJ TO, 0 # i = 0
```

```
LJ T1, 5 # DIM LOOP
```

FOR:

```
BGE TO, T1, EXIT
```

```
ADD TO, TO, 1 # i++
```

```
// DO SOMETHING
```

```
J LOOP # TORNO INDIETRO
```

EXIT: